

Cesar Arturo Almazan Peña

Efren Valle

22170554

16/10/2026

Ciencia de datos

1. Introducción

En este proyecto hacemos un análisis exploratorio de datos (EDA) de un archivo climatológico llamado 25001.txt. La idea es conocer cómo está organizada la información, revisar si faltan datos, completar esos espacios y usar estadísticas y gráficas para entender mejor las variables.

En este documento explicamos el código de la notebook por partes, qué hace cada bloque y qué podemos entender de los resultados.

2. Objetivos

2.1 Objetivo general

Analizar los datos climatológicos para conocer cómo están organizados, revisar su calidad, ver cómo se distribuyen y observar la relación entre las variables.

2.2 Objetivos específicos

- Cargar el archivo de texto y asignar nombres claros a sus columnas.
- Examinar el número de filas, columnas, tipos de datos y registros iniciales y finales.
- Calcular estadísticas descriptivas para las variables numéricas.
- Contar los valores faltantes y calcular el porcentaje que representan.
- Completar los valores faltantes numéricos mediante la media de cada columna y comprobar el resultado.
- Visualizar las distribuciones con histogramas, diagramas de caja, curvas de densidad y gráficas de dispersión.
- Calcular e interpretar la matriz de correlación de Pearson.

3. Datos utilizados

Usamos el archivo 25001.txt. Al leerlo obtuvimos 19,575 registros y cinco columnas: FECHA, PRECIP, EVAP, TMAX y TMIN.

Columna	Significado	Unidad indicada en el proyecto
FECHA	Fecha asociada al registro climatológico.	Fecha
PRECIP	Cantidad de precipitación registrada.	Milímetros (mm)
EVAP	Cantidad de evaporación registrada.	Milímetros (mm)
TMAX	Temperatura máxima registrada.	Grados Celsius (°C)
TMIN	Temperatura mínima registrada.	Grados Celsius (°C)

4. Bibliotecas y carga de datos

Primero importamos las bibliotecas que necesitamos para leer los datos, trabajar con ellos y crear gráficas.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

c

Instrucción	Explicación
import pandas as pd	Importa pandas para leer archivos y trabajar con tablas (DataFrame).
import matplotlib.pyplot as plt	Importa Matplotlib para crear gráficas.
import seaborn as sns	Importa Seaborn para hacer gráficas estadísticas, como el mapa de calor.

4.1 Lectura del archivo

```
df = pd.read_csv(
    "25001.txt",
    sep=r"\s+",
    skiprows=19,
    names=["FECHA", "PRECIP", "EVAP", "TMAX", "TMIN"],
    encoding="latin1",
    na_values=["Nulo"]
)
```

Parámetro	Función
"25001.txt"	Nombre del archivo que se va a leer.
sep=r"\s+"	Indica que los datos están separados por uno o más espacios.
skiprows=19	Salta las primeras 19 líneas, que tienen información antes de la tabla.
names=[...]	Asigna los nombres de las cinco columnas.
encoding="latin1"	Indica la codificación del texto para interpretar correctamente caracteres.
na_values=["Nulo"]	Toma el texto "Nulo" como un dato faltante.

Guardamos la información en la variable df. Esta variable contiene la tabla con sus filas y columnas; en pandas, este tipo de tabla se llama DataFrame.

5. Exploración inicial del DataFrame

5.1 Dimensiones: df.shape

```
df.shape
```

Muestra primero el número de filas y después el de columnas. En este proyecto, (19575, 5) significa que hay 19,575 filas y cinco columnas. shape no lleva paréntesis porque es un atributo.

5.2 Primeros y últimos registros

```
df.head()
df.tail()
```

- df.head() muestra las primeras cinco filas de manera predeterminada.
- df.tail() muestra las últimas cinco filas de manera predeterminada.
- Estas funciones ayudan a comprobar si el archivo se leyó con la estructura esperada.

5.3 Información general: df.info()

```
df.info()
```

Muestra cuántas filas y columnas hay, qué tipo de dato tiene cada columna y cuántos valores no están vacíos. Si una columna tiene menos valores no vacíos que filas, significa que le faltan datos.

5.4 Estadísticas generales: df.describe()

```
df.describe()
```

Genera un resumen de las columnas numéricas: cuántos datos hay, el promedio, la desviación estándar, el mínimo, los cuartiles y el máximo.

Estadística	Qué indica
count	Cantidad de valores no faltantes utilizados en el cálculo.
mean	Promedio aritmético.
std	Indica qué tanto se alejan los datos del promedio.
min / max	Valor mínimo y máximo observado.
25%, 50%, 75%	Cuartiles; 50% corresponde a la mediana.

6. Estadísticas descriptivas calculadas manualmente

```
mediciones = df[["PRECIP", "EVAP", "TMAX", "TMIN"]]

print("Media:")
print(mediciones.mean())

print("\nMediana:")
print(mediciones.median())

print("\nModa:")
print(mediciones.mode().iloc[0])

print("\nDesviación estándar:")
print(mediciones.std())

print("\nVarianza:")
print(mediciones.var())

print("\nRango:")
print(mediciones.max() - mediciones.min())
```

Método	Interpretación
mean()	Calcula la media de cada columna numérica.
median()	Calcula el valor central una vez ordenados los datos.
mode().iloc[0]	Obtiene la primera fila de la tabla de modas; permite mostrar un valor de moda por columna.
std()	Calcula la desviación estándar, una medida de dispersión.

var()	Calcula la varianza, otra medida de dispersión basada en desviaciones al cuadrado.
max() - min()	Calcula el rango, es decir, la diferencia entre el máximo y el mínimo.

La media y la mediana pueden ser diferentes. Por ejemplo, unas lluvias muy altas pueden aumentar el promedio. La moda es el valor que más se repite.

7. Identificación y resumen de datos faltantes

7.1 Conteo por columna

```
print("Valores faltantes por columna:")
print(df.isnull().sum())
```

- isnull() produce una tabla de valores booleanos: True donde el valor falta y False donde existe.
- sum() suma los True por columna y permite contar los valores faltantes.

Columna	Valores faltantes iniciales	Porcentaje aproximado
FECHA	0	0.00 %
PRECIP	41	0.21 %
EVAP	273	1.39 %
TMAX	6	0.03 %
TMIN	7	0.04 %
Total	327	—

7.2 Crear una tabla resumen

```
nulos = df.isnull().sum()
porcentaje = df.isnull().mean() * 100

resumen_nulos = pd.DataFrame({
    "Valores faltantes": nulos,
    "Porcentaje (%)": porcentaje
})

display(resumen_nulos.round(2))
print("Total de datos faltantes:", df.isnull().sum().sum())
```

Expresión	Explicación
df.isnull().sum()	Cuenta los faltantes por columna.
df.isnull().mean()	Calcula la parte de valores faltantes de cada columna; después se multiplica por 100 para obtener el porcentaje.
* 100	Convierte la proporción en porcentaje.
pd.DataFrame({...})	Junta el conteo y el porcentaje en una sola tabla.
round(2)	Deja los valores con dos decimales.
sum().sum()	El primer sum cuenta los faltantes por columna y el

	segundo los suma para obtener el total.

Código adicional: conteo de datos faltantes por fecha

Esta celda identifica las filas que tienen al menos un dato faltante y después cuenta cuántos valores faltan para cada fecha. Se agrega antes de rellenar los datos, porque después de la limpieza ya no se podrían identificar los faltantes originales.

```
registros_faltantes = df[df.isnull().any(axis=1)]

faltantes_por_fecha = registros_faltantes.groupby("FECHA").apply(
    lambda grupo: grupo.isnull().sum().sum()
)

print("Cantidad de registros con datos faltantes:", len(registros_faltantes))
print("\nDatos faltantes por fecha:")
print(faltantes_por_fecha)
```

Explicación del código:

- df.isnull().any(axis=1) revisa cada fila y detecta si tiene por lo menos un valor vacío. Con df[...] guardamos esas filas en registros_faltantes.
- groupby("FECHA") junta los registros según su fecha. La función lambda cuenta los valores faltantes dentro de cada grupo; el primer sum() cuenta por columna y el segundo suma esos conteos.
- len(registros_faltantes) muestra cuántas filas tienen al menos un dato faltante. La tabla faltantes_por_fecha muestra cuántos valores vacíos hay por fecha; una fila puede tener más de un valor faltante.

7.3 Gráfica de barras de valores faltantes

```
nulos.plot(kind="bar")
plt.title("Valores faltantes por variable")
plt.xlabel("Variables")
plt.ylabel("Cantidad de valores faltantes")
plt.show()
```

kind="bar" solicita una gráfica de barras. El eje horizontal muestra las variables y el eje vertical la cantidad de valores faltantes. La barra de EVAP es la más alta porque presenta 273 faltantes.

8. Tratamiento de valores faltantes

Elegimos una forma sencilla de completar los datos: reemplazar cada espacio vacío de las columnas numéricas por el promedio de esa misma columna. Así conservamos los registros, aunque los valores agregados son estimaciones y no mediciones reales.

```
columnas = ["PRECIP", "EVAP", "TMAX", "TMIN"]

for columna in columnas:
    df[columna] = df[columna].fillna(df[columna].mean())

print("Datos faltantes después de rellenar:")
print(df[columnas].isnull().sum())

print("\nTotal de datos faltantes restantes:",
      df[columnas].isnull().sum().sum())
```

Elemento	Explicación

columnas	Lista con los nombres de las cuatro variables que se van a procesar.
for columna in columnas	Va recorriendo la lista y trabaja con una columna a la vez.
df[columna]	Selecciona la columna que corresponde a la iteración actual.
df[columna].mean()	Calcula el promedio de esa columna.
fillna(...)	Rellena los espacios vacíos con el valor indicado.
df[columna] = ...	Guarda el resultado rellenado en la misma columna del DataFrame.
df[columnas].isnull().sum()	Comprueba los faltantes restantes por variable.
df[columnas].isnull().sum().sum()	Suma los conteos restantes para producir un solo total.

Al terminar, comprobamos que ya no quedaban valores faltantes en PRECIP, EVAP, TMAX y TMIN. Sin embargo, usar el promedio no siempre es lo mejor. En PRECIP hay muchos valores cercanos a cero y unos pocos muy altos, por lo que el promedio puede no representar bien los días normales. Más adelante se podría comparar este método con usar la mediana u otros métodos.

9. Histogramas y distribuciones

Los histogramas nos ayudan a ver cómo se reparten los valores y cuáles aparecen con más frecuencia. En la notebook revisamos las cuatro variables numéricas.

- PRECIP: la mayoría de los valores se concentra cerca de cero y existe una cola hacia valores elevados; la asimetría positiva es marcada.
- EVAP: la mayor concentración se observa aproximadamente entre 3 y 5 mm, con una cola hacia valores mayores.
- TMAX: la mayoría de los registros se concentra aproximadamente entre 32 y 39 °C.
- TMIN: presenta una forma irregular y dos concentraciones visibles, aproximadamente alrededor de 9–13 °C y 22–24 °C.

Los histogramas muestran la forma de los datos, pero no podemos decir solo con ellos que los valores muy alejados sean errores.

10. Diagramas de caja (boxplots)

```
df[["PRECIP", "EVAP", "TMAX", "TMIN"]].plot(
    kind="box",
    subplots=True,
    layout=(2, 2),
    figsize=(12, 8)
)
plt.show()
```

Parámetro	Función
kind="box"	Hace diagramas de caja.
subplots=True	Hace una gráfica separada para cada variable.
layout=(2, 2)	Acomoda las gráficas en dos filas y dos columnas.

figsize=(12, 8)	Define el tamaño de la gráfica.

La caja muestra la parte central de los datos y la línea que está dentro representa la mediana. Los bigotes muestran el alcance de los datos según la regla del gráfico. Los puntos que quedan fuera pueden ser valores atípicos, pero no significa que estén mal: podrían ser eventos climatológicos reales.

11. Asimetría de las variables

```
print("Asimetría de las variables:")
print(df[["PRECIP", "EVAP", "TMAX", "TMIN"]].skew())
```

Variable	Valor observado	Interpretación general
PRECIP	7.982828	Asimetría positiva muy marcada; cola hacia valores altos.
EVAP	0.444290	Asimetría positiva leve.
TMAX	-0.823858	Asimetría negativa; cola hacia valores bajos.
TMIN	-0.112764	Cercana a cero; poca asimetría en comparación con las demás.

El método skew() calcula la asimetría. Si el resultado es positivo, la cola de la distribución va más hacia la derecha; si es negativo, va más hacia la izquierda. Si está cerca de cero, hay poca asimetría.

12. Gráficas de densidad

Las curvas de densidad (KDE) muestran una versión más suave de la distribución de los datos. En nuestro análisis, PRECIP se concentra cerca de cero; EVAP tiene su mayor concentración aproximadamente entre 3 y 5 mm; TMAX tiene un pico cerca de 35 °C; y TMIN muestra dos picos principales. A diferencia del histograma, la curva es una estimación suavizada.

13. Gráficas de dispersión por registro

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.scatter(df.index, df["PRECIP"], s=5)
plt.title("Dispersión de la precipitación")
plt.xlabel("Registro")
plt.ylabel("Precipitación (mm)")

plt.subplot(2, 2, 2)
plt.scatter(df.index, df["EVAP"], s=5)
plt.title("Dispersión de la evaporación")
plt.xlabel("Registro")
plt.ylabel("Evaporación (mm)")

plt.subplot(2, 2, 3)
plt.scatter(df.index, df["TMAX"], s=5)
plt.title("Dispersión de la temperatura máxima")
plt.xlabel("Registro")
plt.ylabel("Temperatura (°C)")

plt.subplot(2, 2, 4)
plt.scatter(df.index, df["TMIN"], s=5)
plt.title("Dispersión de la temperatura mínima")
plt.xlabel("Registro")
plt.ylabel("Temperatura (°C)")

plt.tight_layout()
plt.show()
```

Instrucción	Función
plt.figure(figsize=(12, 8))	Crea una figura de 12 por 8 pulgadas.
plt.subplot(2, 2, n)	Selecciona la posición n dentro de una cuadrícula de 2 filas y 2 columnas.
plt.scatter(df.index, df[...], s=5)	Grafica el índice del registro en X y los valores de una variable en Y; s controla el tamaño de los puntos.
plt.xlabel() / plt.ylabel()	Nombran los ejes horizontal y vertical.
plt.title()	Añade el título de cada gráfico.
plt.tight_layout()	Ajusta los espacios para reducir traslapes.
plt.show()	Muestra la figura.

- PRECIP: abundan valores cercanos a cero y aparecen algunos registros muy elevados.
- EVAP: muchos valores se distribuyen aproximadamente entre 2 y 7 mm, con algunas observaciones más altas.
- TMAX: la mayoría de los registros se concentra aproximadamente entre 30 y 40 °C.
- TMIN: presenta una dispersión amplia, con valores desde cerca de 0 hasta aproximadamente 28 °C y algunos extremos.

Importante: el eje X muestra el número de registro y no una fecha ya convertida. Por eso, estas gráficas no deben tomarse como una línea de tiempo hasta convertir y ordenar primero la columna FECHA.

14. Relación entre temperatura máxima y evaporación

```
plt.scatter(df["TMAX"], df["EVAP"])
plt.xlabel("Temperatura máxima (°C)")
plt.ylabel("Evaporación (mm)")
plt.title("Temperatura máxima vs evaporación")
plt.show()
```

En esta gráfica, cada punto representa un registro: la temperatura máxima aparece en el eje X y la evaporación en el eje Y. En general, los puntos suben hacia la derecha, lo que indica que las temperaturas máximas más altas suelen aparecer junto con una evaporación mayor. Esto coincide con la correlación positiva, aunque no demuestra que una variable cause la otra.

15. Matriz de correlación

```
variables = df[["PRECIP", "EVAP", "TMAX", "TMIN"]]
correlacion = variables.corr()

plt.figure(figsize=(8, 6))
sns.heatmap(
    correlacion,
    annot=True,
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    fmt=".2f"
)
plt.title("Matriz de correlación de las variables climatológicas")
plt.show()
```

Parámetro	Función
variables.corr()	Calcula la correlación entre las columnas numéricas seleccionadas.
sns.heatmap()	Muestra la matriz con colores en un mapa de calor.
annot=True	Muestra los números dentro de las celdas.
cmap="coolwarm"	Elige los colores del mapa.
vmin=-1, vmax=1	Fija el mínimo en -1 y el máximo en +1.
fmt=".2f"	Muestra los números con dos decimales.

Par de variables	Correlación aproximada	Interpretación
EVAP – TMAX	0.63	Positiva moderada a fuerte.
TMAX – TMIN	0.40	Positiva moderada.
PRECIP – TMIN	0.24	Positiva débil.
EVAP – TMIN	0.14	Positiva muy débil.
PRECIP – EVAP	-0.07	Negativa muy débil.
PRECIP – TMAX	-0.06	Negativa muy débil.

La correlación de Pearson va de -1 a +1. Si es positiva, las variables suelen aumentar juntas; si es negativa, suelen moverse en direcciones opuestas; y si está cerca de cero, hay poca relación lineal. Una correlación alta no demuestra que una variable cause la otra ni significa automáticamente que haya datos duplicados.

16. Relación entre correlación e imputación

La matriz de correlación nos ayuda a ver cómo se relacionan las variables, pero no demuestra por sí sola que rellenar con el promedio sea la mejor opción. En este proyecto usamos la media porque es un método sencillo para completar las columnas numéricas y conservar los registros.

La asimetría positiva marcada de PRECIP indica que la media puede ser poco representativa de los días típicos. Por ello, la imputación debe presentarse como una decisión práctica de limpieza, no como una conclusión demostrada por la matriz. En un análisis posterior podrían compararse la mediana u otros métodos que consideren la fecha y las demás variables.

17. Conclusiones

Con el análisis pudimos revisar cómo está organizado el archivo 25001.txt, encontrar 327 valores faltantes, observar las distribuciones y estudiar las relaciones entre las variables. Después de rellenar los faltantes con el promedio de cada columna, comprobamos que quedaron cero valores faltantes en PRECIP, EVAP, TMAX y TMIN.

Las gráficas mostraron que la precipitación tiene muchos valores cerca de cero y algunos muy altos, que la evaporación se concentra sobre todo entre 3 y 5 mm, que la temperatura máxima se concentra alrededor de 35 °C y que la temperatura mínima tiene dos concentraciones principales. La correlación más alta fue entre EVAP y TMAX (aproximadamente 0.63), seguida de TMAX y TMIN (aproximadamente 0.40).

Los valores que rellenamos son estimaciones, no mediciones reales. La correlación muestra relaciones entre variables, pero no demuestra que una cause la otra ni que haya datos duplicados. En el futuro sería bueno revisar los valores atípicos con su fecha y contexto, y comprobar si usar el promedio fue adecuado, sobre todo para PRECIP.

18. Herramientas utilizadas

- Python para ejecutar el código.
- pandas para leer, organizar, resumir y limpiar los datos.
- Matplotlib para crear gráficas.
- Seaborn para la visualización de la matriz de correlación.
- Notebook de Python como entorno de ejecución.

19. Repositorio

Repositorio GitHub del proyecto:

https://github.com/Caap14/Ciencia_De_Datos_AlmazanValle

Anexo. Guía rápida de funciones

Función o atributo	Uso breve
pd.read_csv()	Leer el archivo de texto como tabla.
df.shape	Consultar número de filas y columnas.

df.head() / df.tail()	Ver primeras o últimas filas.
df.info()	Ver tipos y valores no nulos.
df.describe()	Consultar estadísticas descriptivas.
mean() / median() / mode()	Media, mediana y moda.
isnull().sum()	Contar datos faltantes.
fillna()	Reemplazar valores faltantes.
skew()	Calcular asimetría.
plot(kind='box')	Crear diagramas de caja.
plt.scatter()	Crear gráficas de dispersión.
corr()	Calcular correlaciones.
sns.heatmap()	Mostrar la matriz de correlación con colores.
plt.show()	Mostrar una gráfica.
iloc	Seleccionar filas/columnas por posición numérica.